

Slowly Changing Dimensions & Snapshots

How to preserve dimension history in dbt — and why we went beyond native snapshots.

The Problem: History That Disappears

Contact 42 moves from **Lead** → **Opportunity** on March 15. Your nightly dbt run fires.

BEFORE — March 14

contact_id	email	pipeline_stage
42	anna@example.com	lead
43	ben@example.com	opportunity

AFTER — March 15 (post-run)

contact_id	email	pipeline_stage
42	anna@example.com	opportunity
43	ben@example.com	opportunity

Stakeholder question: "How many contacts were in the Lead stage at the end of Q1?"

You cannot answer this. The warehouse no longer contains that information.

SCD Type 2: Keep Both Rows

When a tracked attribute changes, **close the old row** and **insert a new row**. Never overwrite.

contact_id	email	pipeline_stage	valid_from	valid_to
42	anna@example.com	lead	2024-01-10	2024-03-15
42	anna@example.com	opportunity	2024-03-15	NULL ← current
43	ben@example.com	opportunity	2024-01-10	NULL ← current

NULL in valid_to = this row is current. The contact is in this state right now.

Non-NULL valid_to = this row is historical. The contact was in this state between the two dates.

dbt Snapshot Syntax

```
{% snapshot snap_contacts %}

{{
    config(
        target_schema = 'snapshots',
        unique_key     = 'contact_id',
        strategy       = 'timestamp',
        updated_at      = 'updated_at'
    )
}}

SELECT
    contact_id,
    email,
    pipeline_stage,
    updated_at
FROM {{ ref('stg_hubspot__contacts') }}

{% endsnapshot %}
```

Lives in

snapshots/ folder — not models/

Run with

dbt snapshot — not dbt run

SELECT is

Plain SQL — dbt manages the SCD2 rows from it

Snapshot Strategies: `timestamp` vs `check`

	<code>timestamp</code>	<code>check</code>
How it detects changes	Compares <code>updated_at</code> column	Hashes listed column values
Speed	Fast — single column compare	Slower — hashes all <code>check_cols</code> rows
Requirement	Reliable <code>updated_at</code> in source	No timestamp requirement
Risk	Misses changes if <code>updated_at</code> not bumped	None — always detects value changes

`timestamp strategy`

```
config(  
  strategy = 'timestamp',
```

`check strategy`

```
config(  
  strategy = 'check',
```

Snapshot Metadata Columns

Column	What it holds	Meaning of NULL
dbt_valid_from	Timestamp when this row version became active	Never NULL
dbt_valid_to	Timestamp when this row version was superseded	NULL = this is the current row
dbt_scd_id	Unique identifier for this row version (SHA1 hash)	Never NULL
dbt_updated_at	The source's updated_at at insert time	Never NULL

Point-in-time query – "What was contact 42's stage on March 10?"

```
SELECT contact_id, email, pipeline_stage
FROM snapshots.snap_contacts
WHERE dbt_valid_from ≤ '2024-03-10'
AND (dbt_valid_to > '2024-03-10' OR dbt_valid_to IS NULL)
```

The `OR dbt_valid_to IS NULL` is not optional — it includes the current row, which has no end date.

Two Runs — What the Table Looks Like

After first `dbt snapshot`

contact_id	stage	valid_from	valid_to
1	lead	Jan 10	NULL
2	opportunity	Jan 10	NULL
3	lead	Jan 10	NULL

3 rows. All current. All valid_to = NULL.

After second run (contact 1 changed stage)

contact_id	stage	valid_from	valid_to
1	lead	Jan 10	Mar 15
1	opportunity	Mar 15	NULL
2	opportunity	Jan 10	NULL
3	lead	Jan 10	NULL

4 rows. Contact 1 has two. Contacts 2 and 3 unchanged.

dbt closed the old row (set valid_to = Mar 15) and inserted a new current row. Contacts 2 and 3 were not touched — their `updated_at` didn't change.

The Surrogate Key Stability Problem

Native snapshots work correctly. But they have one specific weakness: **surrogate key regeneration on full refresh.**

What `dbt_scd_id` is

SHA1 hash of `unique_key + dbt_valid_from`. Generated at insert time. Used as FK in downstream fact tables.

```
-- fct_deal references the snapshot row
SELECT
  deal_id,
  contact_scd_id,    -- = snap_contacts.dbt_scd_id
  deal_amount
FROM fct_deal
```

What happens on `dbt snapshot --full-refresh`

1. Snapshot table is dropped and rebuilt from scratch.
2. All rows re-inserted with new timestamps.
3. `dbt_valid_from` changes → `dbt_scd_id` changes.
4. Every FK in `fct_deal` now points to nothing.

Result: A full refresh silently breaks referential integrity across your entire Silver layer. No error. Wrong joins.

The scd2_merge Pattern: Stable Keys

The fix: compute the surrogate key from **source data values**, not from insert timestamps.

```
WITH source AS (  
  SELECT  
    contact_id,  
    email,  
    pipeline_stage,  
    -- Hash derived from source data – same result on every run, including full refresh  
    md5(contact_id || '|' || pipeline_stage) AS contact_scd_key,  
    -- Row hash – detects any change in tracked columns  
    md5(email || '|' || pipeline_stage) AS row_hash,  
    updated_at  
  FROM {{ ref('stg_hubspot_contacts') }}  
)
```

contact_scd_key is derived from contact_id + pipeline_stage. Run it 100 times — same result. Full refresh → same keys. FK references stay valid.

row_hash detects changes. If email or pipeline_stage changes, row_hash changes → a new SCD2 row is inserted. You control which columns trigger versioning.

Exercise — Build and Operate a Snapshot (25 min)

Goal: Create a snapshot for HubSpot contacts, observe SCD2 in action by simulating a data change, then answer a historical question using the snapshot table.

Task 1 Create snapshots/snap_contacts.sql

Use `timestamp` strategy, `unique_key = 'contact_id'`, `updated_at = 'updated_at'`, `target_schema = 'snapshots'`. Track: `contact_id`, `first_name`, `last_name`, `email`, `pipeline_stage`, `updated_at`.

Task 2 Build the initial snapshot

Run `dbt snapshot`. Verify: row count matches `raw_contacts`. All `dbt_valid_to = NULL`.

Task 3 Simulate a change

Edit `seeds/raw_contacts.csv` — change `contact_id 3`'s email and bump `updated_at` by one day. Run `dbt seed` then `dbt snapshot`. Verify: contact 3 now has two rows.

Bonus Answer a historical question

Key Takeaways



dbt native snapshots implement SCD2 correctly

The `timestamp` strategy detects changes via `updated_at`. Metadata columns (`dbt_valid_from` , `dbt_valid_to`) enable point-in-time queries. For many projects, native snapshots are the right choice.



Surrogate key stability is the one reason we opted out

`dbt_scd_id` regenerates on full refresh because it includes the insert timestamp. When downstream fact tables use it as a foreign key, a full refresh silently breaks all joins. This is a production data quality risk that doesn't surface as an error.



The `scd2_merge` pattern computes keys from data, not from timestamps

Hash keys are derived from source data values in the source CTE. Same source data always produces the same key — full refresh safe. You also control which columns trigger versioning via the `row_hash`.

MODULE 10 COMPLETE

Next: Module 11

Advanced Testing and Data Contracts

Prep Q1: What does `dbt_valid_to = NULL` mean in a snapshot table?

Prep Q2: When would you choose the check strategy over timestamp?

Prep Q3: Why does `dbt snapshot --full-refresh` break downstream FKs?

Prep Q4: In `scd2_merge`, where is the surrogate key hash computed and why?